

Assignment Day 25 - Proses ETL dan Data Warehouse E-Commerce Transaction

M Aqila Musyaffa
DABI-22

Apa itu ETL?



ETL (Extract, Transform, Load) adalah suatu proses integrasi data yang digunakan untuk mengambil, menyatukan, membersihkan, dan menyimpan data.

Extract

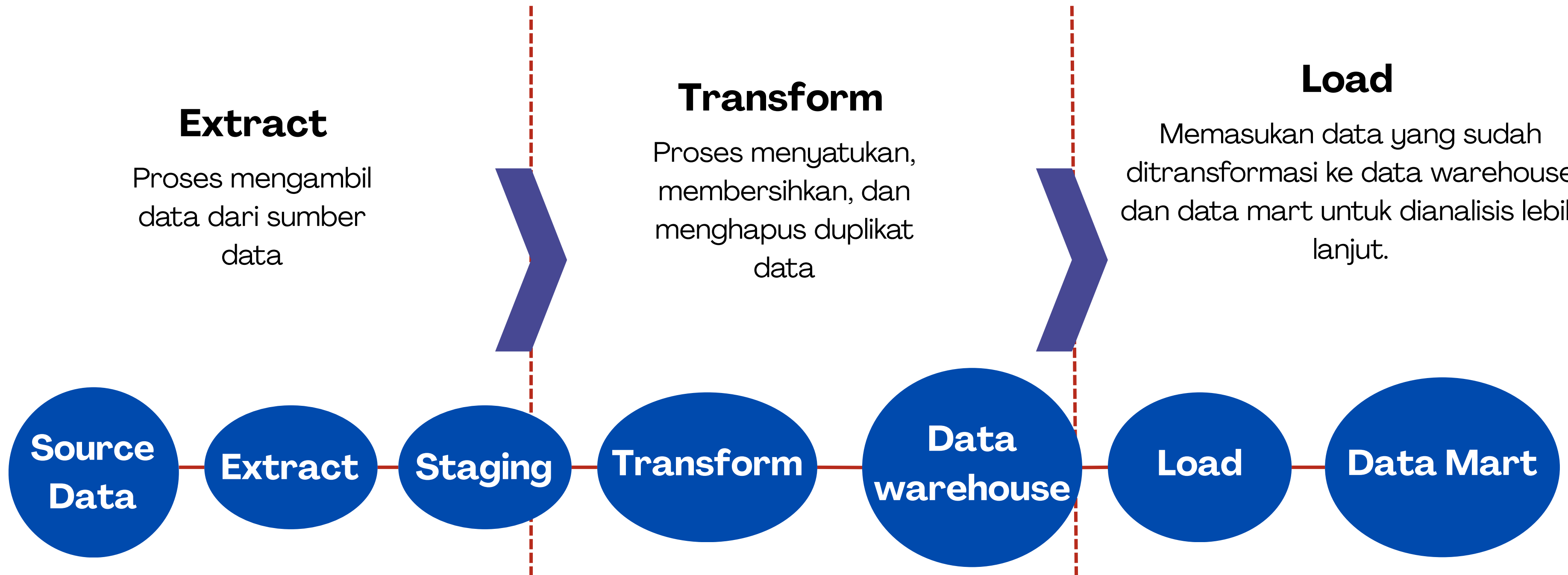
Proses mengambil data dari sumber data

Transform

Proses menyatukan, membersihkan, dan menghapus duplikat data

Load

Memasukan data yang sudah ditransformasi ke data warehouse dan data mart untuk dianalisis lebih lanjut.



Apa itu ETL?



ETL (Extract, Transform, Load) adalah suatu proses integrasi data yang digunakan untuk mengambil, menyatukan, membersihkan , dan menyimpan data.

Extract

Proses mengambil data dari sumber data

query :

```
-- Step 1: Load Data into Staging (stg)
CREATE TABLE IF NOT EXISTS stg.stg_ecommerce_transaction AS
SELECT *, CURRENT_TIMESTAMP AS last_update FROM public.ecommerce_transaction;
TRUNCATE TABLE stg.stg_ecommerce_transaction;
INSERT INTO stg.stg_ecommerce_transaction
SELECT *, CURRENT_TIMESTAMP AS last_update FROM public.ecommerce_transaction;
```

Transform

Proses menyatukan, membersihkan, dan menghapus duplikat data

```
-- Step 2: Load Data into Dim_Product
REATE TABLE IF NOT EXISTS dwh.dim_ecommerce_product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(255),
  product_category VARCHAR(50),
  product_price FLOAT4,
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
RUNCATE TABLE dwh.dim_ecommerce_product;
INSERT INTO dwh.dim_ecommerce_product (product_id, product_name, product_category, product_price, last_update)
ELECT src.product_id, src.product_name, src.product_category, src.product_price, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.product_id, src.product_name, src.product_category, src.product_price;

-- Step 3: Load Data into Dim_Store
REATE TABLE IF NOT EXISTS dwh.dim_ecommerce_store (
  store_id INT PRIMARY KEY,
  store_name VARCHAR(255),
  store_phone VARCHAR(50),
  store_city VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
RUNCATE TABLE dwh.dim_ecommerce_store;
INSERT INTO dwh.dim_ecommerce_store (store_id, store_name, store_phone, store_city, last_update)
ELECT src.store_id, src.store_name, src.store_phone, src.store_city, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.store_id, src.store_name, src.store_phone, src.store_city;

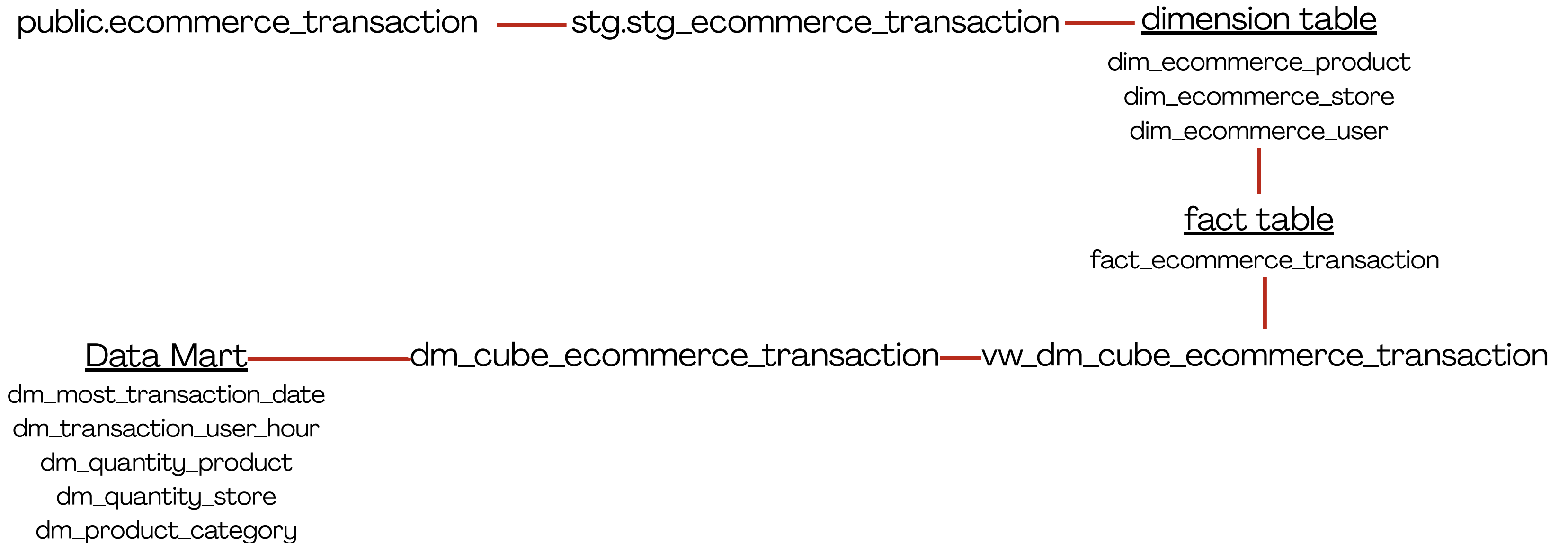
-- Step 4: Load Data into Dim_User
REATE TABLE IF NOT EXISTS dwh.dim_ecommerce_user (
  user_id INT PRIMARY KEY,
  user_name VARCHAR(255),
  user_email VARCHAR(255),
  user_phone VARCHAR(50),
  user_gender VARCHAR(50),
  user_age INT,
  user_city VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
RUNCATE TABLE dwh.dim_ecommerce_user;
INSERT INTO dwh.dim_ecommerce_user (user_id, user_name, user_email, user_phone, user_gender, user_age, user_city, last_update)
ELECT src.user_id, src.user_name, src.user_email, src.user_phone, src.user_gender, src.user_age, src.user_city, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.user_id, src.user_name, src.user_email, src.user_phone, src.user_gender, src.user_age, src.user_city;

-- Step 5: Load Data into Fact_Sales
REATE TABLE IF NOT EXISTS dwh.fact_ecommerce_transaction (
  sale_id INT PRIMARY KEY,
  transaction_date DATE,
  transaction_time TIME,
  store_id INT,
  user_id INT,
  product_id INT,
  quantity INT,
  total_price FLOAT,
  payment_method VARCHAR(50),
  transaction_status VARCHAR(50),
  shipping_method VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO dwh.fact_ecommerce_transaction (sale_id, transaction_date, transaction_time, store_id, user_id, product_id,
ELECT
  src.transaction_id AS sale_id,
  src.transaction_time::DATE AS transaction_date,
  src.transaction_time::TIME AS transaction_time,
  src.store_id,
  src.user_id,
  src.product_id,
  src.quantity,
  src.total_price,
  src.payment_method,
  src.transaction_status,
  src.shipping_method,
  CURRENT_TIMESTAMP AS last_update
FROM stg.stg_ecommerce_transaction AS src
WHERE NOT EXISTS (
  SELECT 1 FROM dwh.fact_ecommerce_transaction AS fact
  WHERE fact.sale_id = src.transaction_id
);
```

Load

Memasukan data yang sudah ditransformasi ke data warehouse dan data mart untuk dianalisis lebih lanjut.

Stored Procedure menjalankan :



Step Extract



Data diambil dari sumber

Data Pusat : public.ecommerce_transaction

```
CREATE TABLE public.ecommerce_transaction (  
  transaction_id int4 NULL,  
  transaction_time information_schema."time_stamp" NULL,  
  user_id int4 NULL,  
  user_name varchar(50) NULL,  
  user_email varchar(50) NULL,  
  user_phone varchar(50) NULL,  
  user_gender varchar(50) NULL,  
  user_age int4 NULL,  
  user_city varchar(50) NULL,  
  store_id int4 NULL,  
  store_name varchar(50) NULL,  
  store_phone varchar(50) NULL,  
  store_city varchar(50) NULL,  
  product_id int4 NULL,  
  product_name varchar(50) NULL,  
  product_category varchar(50) NULL,  
  product_price float4 NULL,  
  quantity int4 NULL,  
  total_price float4 NULL,  
  payment_method varchar(50) NULL,  
  transaction_status varchar(50) NULL,  
  shipping_method varchar(50) NULL  
);
```


OLTP

Lalu disalindan masuk ke tahap staging :

```
-- Step 1: Load Data into Staging (stg)  
CREATE TABLE IF NOT EXISTS stg.stg_ecommerce_transaction AS  
SELECT *, CURRENT_TIMESTAMP AS last_update FROM public.ecommerce_transaction;  
TRUNCATE TABLE stg.stg_ecommerce_transaction;  
INSERT INTO stg.stg_ecommerce_transaction  
SELECT *, CURRENT_TIMESTAMP AS last_update FROM public.ecommerce_transaction;
```

Fungsi Staging :

1. Menyimpan data sementara
2. Memisahkan data source dengan data yang akan diolah (data warehouse)
3. Untuk mempermudah proses transformasi

Step Transform



Data pada staging diubah menjadi tabel fact dan tabel dimension

Dimension Table :

1. dwh.dim_ecommerce_product
2. dwh.dim_ecommerce_store
3. dwh.dim_ecommerce_user

```
-- Step 2: Load Data into Dim_Product
REATE TABLE IF NOT EXISTS dwh.dim_ecommerce_product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(255),
  product_category VARCHAR(50),
  product_price FLOAT4,
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
TRUNCATE TABLE dwh.dim_ecommerce_product;
INSERT INTO dwh.dim_ecommerce_product (product_id, product_name, product_category, product_price, last_update)
SELECT src.product_id, src.product_name, src.product_category, src.product_price, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.product_id, src.product_name, src.product_category, src.product_price;

-- Step 3: Load Data into Dim_Store
REATE TABLE IF NOT EXISTS dwh.dim_ecommerce_store (
  store_id INT PRIMARY KEY,
  store_name VARCHAR(255),
  store_phone VARCHAR(50),
  store_city VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
TRUNCATE TABLE dwh.dim_ecommerce_store;
INSERT INTO dwh.dim_ecommerce_store (store_id, store_name, store_phone, store_city, last_update)
SELECT src.store_id, src.store_name, src.store_phone, src.store_city, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.store_id, src.store_name, src.store_phone, src.store_city;

-- Step 4: Load Data into Dim_User
REATE TABLE IF NOT EXISTS dwh.dim_ecommerce_user (
  user_id INT PRIMARY KEY,
  user_name VARCHAR(255),
  user_email VARCHAR(255),
  user_phone VARCHAR(50),
  user_gender VARCHAR(50),
  user_age INT,
  user_city VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
TRUNCATE TABLE dwh.dim_ecommerce_user;
INSERT INTO dwh.dim_ecommerce_user (user_id, user_name, user_email, user_phone, user_gender, user_age, user_city, last_update)
SELECT src.user_id, src.user_name, src.user_email, src.user_phone, src.user_gender, src.user_age, src.user_city, CURRENT_TIMESTAMP
FROM stg.stg_ecommerce_transaction AS src
GROUP BY src.user_id, src.user_name, src.user_email, src.user_phone, src.user_gender, src.user_age, src.user_city;
```

Tabel dimensi menjadi **referensi detail** dari transaksi pada **fact table**

Fact table :

fact_ecommerce_transaction

```
-- Step 5: Load Data into Fact_Sales
REATE TABLE IF NOT EXISTS dwh.fact_ecommerce_transaction (
  sale_id INT PRIMARY KEY,
  transaction_date DATE,
  transaction_time TIME,
  store_id INT,
  user_id INT,
  product_id INT,
  quantity INT,
  total_price FLOAT,
  payment_method VARCHAR(50),
  transaction_status VARCHAR(50),
  shipping_method VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO dwh.fact_ecommerce_transaction (sale_id, transaction_date, transaction_time, store_id, user_id, product_id,
SELECT
  src.transaction_id AS sale_id,
  src.transaction_time::DATE AS transaction_date,
  src.transaction_time::TIME AS transaction_time,
  src.store_id,
  src.user_id,
  src.product_id,
  src.quantity,
  src.total_price,
  src.payment_method,
  src.transaction_status,
  src.shipping_method,
  CURRENT_TIMESTAMP AS last_update
FROM stg.stg_ecommerce_transaction AS src
WHERE NOT EXISTS (
  SELECT 1 FROM dwh.fact_ecommerce_transaction AS fact
  WHERE fact.sale_id = src.transaction_id
);
```

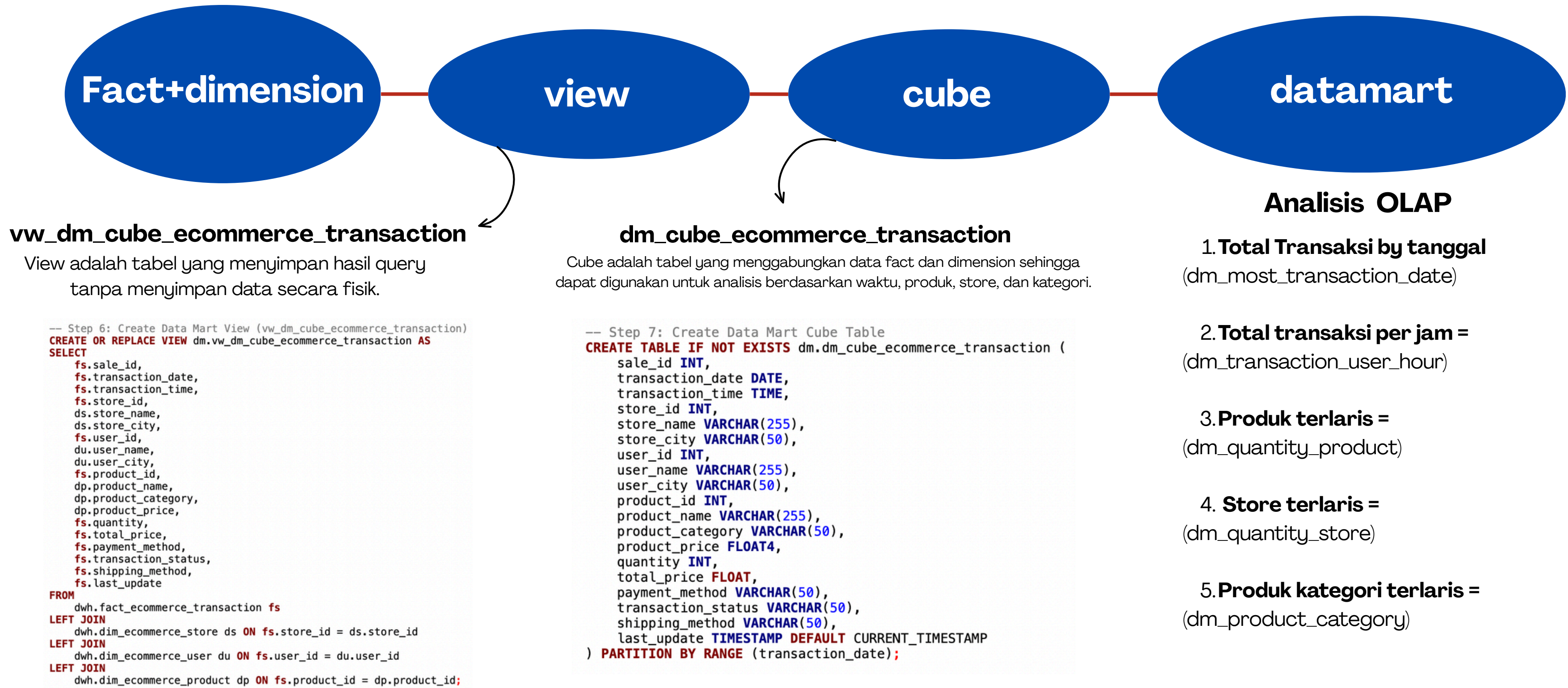
Fungsi Utama dari fact table adalah **menyimpan metrik** hasil dari proses bisnis seperti

- quantity
- total_price
- payment_method
- shipping_method
- etc

Step Load



Memuat data yang telah ditransformasi ke dalam Data Mart untuk kebutuhan analisis bisnis.



Step Load



DATA MART

1. Total Transaksi by tanggal = (dm_most_transaction_date)

```
-- Step 10: Refresh Data in Most Transactions by Date Data Mart
-- Automation: Create Table
CREATE TABLE IF NOT EXISTS dm.dm_most_transaction_date AS
SELECT transaction_date, COUNT(*) AS total_transactions, CURRENT_TIMESTAMP AS last_update
FROM dm.dm_cube_ecommerce_transaction
GROUP BY transaction_date
ORDER BY total_transactions DESC;

-- Automation: Truncate --> SCD Type 1
TRUNCATE TABLE dm.dm_most_transaction_date;

-- Automation: Inserting New Data
INSERT INTO dm.dm_most_transaction_date
SELECT transaction_date, COUNT(*) AS total_transactions, CURRENT_TIMESTAMP AS last_update
FROM dm.dm_cube_ecommerce_transaction
GROUP BY transaction_date
ORDER BY total_transactions DESC;
```

2. Total transaksi per jam = (dm_transaction_user_hour)

```
-- Step 11: Total Transaction and Total User per hour
-- Automation: Create Table
create table if not exists dm.dm_transaction_user_hour(
    transaction_hour int,
    total_transactions int,
    total_users int,
    last_update timestamp default current_timestamp
);
-- Automation: Truncate --> SCD Type 1
truncate table dm.dm_transaction_user_hour;
-- Automation: Inserting New Data
insert into dm.dm_transaction_user_hour (
    transaction_hour,
    total_transactions,
    total_users,
    last_update
)
select
    extract(hour from transaction_time)::int as transaction_hour,
    count(sale_id) as total_transactions,
    count(distinct user_id) as total_users,
    current_timestamp
from dm.dm_cube_ecommerce_transaction
group by extract(hour from transaction_time)
order by transaction_hour;
```

3. Produk terlaris = (dm_quantity_product)

```
-- Step 12: Total Quantity by Product ID
-- Automation: Create Table
create table if not exists dm.dm_quantity_product(
    product_id int,
    product_name varchar(255),
    total_quantity int,
    last_update timestamp default current_timestamp
);
-- Automation: Truncate --> SCD Type 1
truncate table dm.dm_quantity_product;

-- Automation: Inserting New Data
insert into dm.dm_quantity_product (
    product_id,
    product_name,
    total_quantity,
    last_update
)
select
    product_id,
    product_name,
    sum(quantity) as total_quantity,
    current_timestamp
from dm.dm_cube_ecommerce_transaction
group by
    product_id,
    product_name
order by total_quantity desc;
```

4. Store terlaris = (dm_quantity_store)

```
-- Step 13: Total Quantity by Store ID
-- Automation: Create Table
create table if not exists dm.dm_quantity_store(
    store_id int,
    store_name varchar(255),
    total_quantity int,
    last_update timestamp default current_timestamp
);
-- Automation: Truncate --> SCD Type 1
truncate table dm.dm_quantity_store;

-- Automation: Inserting New Data
insert into dm.dm_quantity_store (
    store_id,
    store_name,
    total_quantity,
    last_update
)
select
    store_id,
    store_name,
    sum(quantity) as total_quantity,
    current_timestamp
from dm.dm_cube_ecommerce_transaction
group by
    store_id,
    store_name
order by total_quantity desc;
```

5. Produk kategori terlaris = (dm_product_category)

```
-- Step 14: Additional DM (rename to your datamart)
-- Automation: Create Table
create table if not exists dm.dm_product_category(
    product_category varchar(50),
    total_transactions int,
    total_quantity int,
    total_revenue float,
    last_update timestamp default current_timestamp
);
-- Automation: Truncate --> SCD Type 1
truncate table dm.dm_product_category;

-- Automation: Inserting New Data
insert into dm.dm_product_category (
    product_category,
    total_transactions,
    total_quantity,
    total_revenue,
    last_update
)
select
    product_category,
    count(sale_id) as total_transactions,
    sum(quantity) as total_quantity,
    sum(total_price) as total_revenue,
    current_timestamp
from dm.dm_cube_ecommerce_transaction
group by product_category
order by total_revenue desc;
```


Apa itu Stored Procedure?



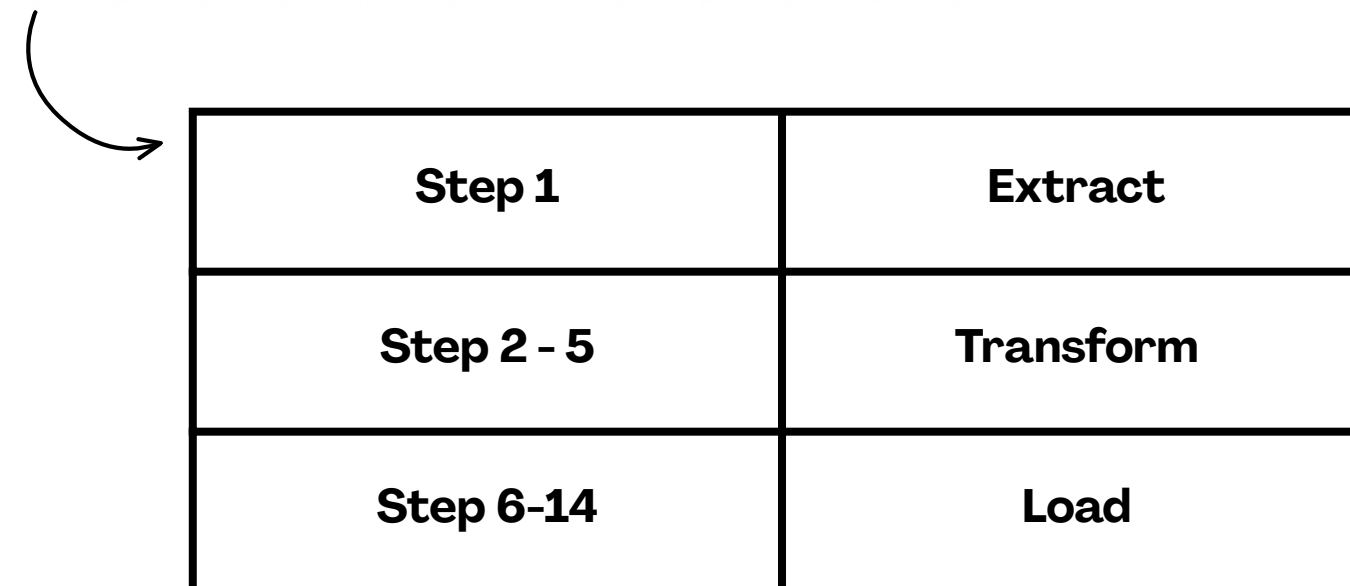
Kumpulan perintah SQL yang disimpan di database dan dapat dijalankan secara otomatis menggunakan satu perintah.

Query :

```
CREATE OR REPLACE PROCEDURE dwh.generate_ecommerce_transaction()  
LANGUAGE plpgsql  
AS $procedure$  
BEGIN
```

dengan memanggil :

```
call dwh.generate_ecommerce_transaction();
```



Step 1	Extract
Step 2 - 5	Transform
Step 6-14	Load

Seluruh proses mulai dari pengambilan data pada source, pembentukan staging, dimension, fact, hingga pembuatan data mart dibungkus dalam satu procedure bernama **dwh.generate_ecommerce_transaction**. Dengan demikian **proses refresh data** dapat dilakukan hanya dengan menjalankan perintah **CALL dwh.generate_ecommerce_transaction()** sehingga lebih efisien dan mengurangi risiko kesalahan manual.

Data Mart

Implementasi Partitioning



Partitioning adalah teknik membagi satu tabel besar menjadi beberapa tabel kecil berdasarkan kriteria tertentu.

Query :

```
-- Step 7: Create Data Mart Cube Table
CREATE TABLE IF NOT EXISTS dm.dm_cube_ecommerce_transaction (
  sale_id INT,
  transaction_date DATE,
  transaction_time TIME,
  store_id INT,
  store_name VARCHAR(255),
  store_city VARCHAR(50),
  user_id INT,
  user_name VARCHAR(255),
  user_city VARCHAR(50),
  product_id INT,
  product_name VARCHAR(255),
  product_category VARCHAR(50),
  product_price FLOAT4,
  quantity INT,
  total_price FLOAT,
  payment_method VARCHAR(50),
  transaction_status VARCHAR(50),
  shipping_method VARCHAR(50),
  last_update TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) PARTITION BY RANGE (transaction_date);
```

tabel **dm.dm_cube_ecommerce_transaction** dibagi berdasarkan **tanggal**

Membuat Partitioning secara otomatis setiap hari :

```
-- Step 8: Create Daily Partitions (2025-01-01 to 2025-03-28)
DO $$
DECLARE d DATE := '2025-01-01';
BEGIN
  WHILE d <= '2025-03-28' LOOP
    EXECUTE format(
      'CREATE TABLE IF NOT EXISTS dm.dm_cube_ecommerce_transaction_%s
      PARTITION OF dm.dm_cube_ecommerce_transaction
      FOR VALUES FROM (%L) TO (%L);',
      to_char(d, 'YYYYMMDD'), d, d + INTERVAL '1 day'
    );
    d := d + INTERVAL '1 day';
  END LOOP;
END $$;
```

Sistem hanya **membaca partisi yang relevan** berdasarkan kriteria query (seperti tanggal atau wilayah), sehingga **tidak perlu memindai seluruh tabel** yang akan berpengaruh pada efisiensi pengolahan data.

Manfaat Partitioning :

1. **Query lebih cepat** karena hanya akan membaca partition dan tidak perlu dibaca seluruh tabel
2. **Mengurangi beban database** karena dengan partitioning data akan terbagi lebih kecil
3. Karena **data akan selalu bertambah setiap hari**, Partitioning penting untuk digunakan

Hasil, Manfaat, dan Kesimpulan



Total transaksi per jam = (dm_transaction_user_hour)

```
--melihat data berdasarkan jam transaksi  
SELECT *  
FROM dm.dm_transaction_user_hour  
ORDER BY transaction_hour;
```

Produk terlaris = (dm_quantity_product)

```
--melihat product terlaris  
SELECT *  
FROM dm.dm_quantity_product  
ORDER BY total_quantity DESC  
LIMIT 10;
```

Store terlaris = (dm_quantity_store)

```
-- cek kategori  
SELECT *  
FROM dm.dm_product_category  
ORDER BY total_revenue DESC;  
----
```

Produk kategori terlaris = (dm_product_category)

```
--melihat product terlaris berdasarkan store  
SELECT *  
FROM dm.dm_quantity_store  
ORDER BY total_quantity DESC  
LIMIT 10;
```

Kesimpulan

Proses ETL berhasil mengintegrasikan data transaksi ke dalam Data Warehouse dan Data Mart yang terstruktur serta **siap digunakan untuk analisis bisnis**. Data mentah berhasil ditransformasikan menjadi **informasi yang berkualitas**, sementara **Stored Procedure** mengotomatisasi seluruh proses ETL sehingga pemuatan dan pembaruan data dapat dilakukan secara **efisien, konsisten, dan minim kesalahan**.

Manfaat

1. Mendukung Analisis dan Keputusan Bisnis
2. Kualitas dan Konsistensi Data
3. Efisiensi Operasional
4. Skalabilitas

Assignment



Terima kasih!

**M Aqila Musyaffa
DABI 22**